Mathematics Stack Exchange is a question and answer site for people studying math at any level and professionals in related fields. It only takes a minute to sign up.

Sign up to join this community

Anybody can ask a question

Anybody can answer

The best answers are voted up and rise to the top

MATHEMATICS

# Best Fitting Plane given a Set of Points

Asked 12 years, 3 months ago Modified 1 year, 4 months ago Viewed 129k times

56

Nothing more to explain. I just don't know how to find the best fitting plane given a set of $N$ points in a $3D$ space. I then have to write the corresponding algorithm. Thank you ;)

linear-algebra

Share Cite Follow

edited Nov 28, 2017 at 5:40
Siong Thye Goh
150k 20 88 149

asked Jan 15, 2012 at 16:15
G4bri3l
693 1 6 9

5 There is plenty more to explain. There are many different measures of how well a plane fits given data, and different measures give rise to different "best" fitting planes. So you had best tell us what you have in mind as your measure of how well a given plane fits some given data. – Gerry Myerson Jan 15, 2012 at 21:02

I'm sorry but I wish I could tell you more. But I know just a bit. Let's say that the set of Points I have (over a 100) already look like a plane, I mean, they are displayed as a plane but not perfectly. "Obtain the symmetry plane A by fitting it on the set of points B.." That's all I have to do. They don't say anything more. – G4bri3l Jan 16, 2012 at 9:20

1 You hadn't mentioned the symmetry part before -- do you know what that's referring to? – joriki Jan 16, 2012 at 9:53

5 Let's look at a simpler problem. Say you have a bunch of points in 2 dimensions that almost lie along a line, but not quite, and you want to find the line that fits those points the best. You could draw a line, then draw vertical line segments from each point to the line, and add up the lengths of all those line segments, and ask for the line that makes that sum as small as possible. But you could draw horizontal line segments instead, and you might get a different answer by minimizing the sum of those lengths. Or you could draw line segments perpendicular to the line. Continued... – Gerry Myerson Jan 16, 2012 at 11:23

2 ...Instead of just adding up the lengths of the line segments, you could add up the squares of the lengths - may seem like a strange idea, but it's very often a good one in this kind of problem. So you have all

those choices, just for drawing a line in 2 dimensions; there are even more choices for a plane in 3. That's why you really have to know what someone means when they ask you to fit a plane to some points. – Gerry Myerson Jan 16, 2012 at 11:26

## 5 Answers

Sorted by: Highest score (default)

49

Subtract out the centroid, form a $3 \times N$ matrix $\mathbf{X}$ out of the resulting coordinates and calculate its singular value decomposition. The normal vector of the best-fitting plane is the left singular vector corresponding to the least singular value. See this answer for an explanation why this is numerically preferable to calculating the eigenvector of $\mathbf{X}\mathbf{X}^\top$ corresponding to the least eigenvalue.

Here's a Python implementation, as requested:

```
import numpy as np

# generate some random test points
m = 20 # number of points
delta = 0.01 # size of random displacement
origin = np.random.rand(3, 1) # random origin for the plane
basis = np.random.rand(3, 2) # random basis vectors for the plane
coefficients = np.random.rand(2, m) # random coefficients for points on the plane

# generate random points on the plane and add random displacement
points = basis @ coefficients \
         + np.tile(origin, (1, m)) \
         + delta * np.random.rand(3, m)

# now find the best-fitting plane for the test points

# subtract out the centroid and take the SVD
svd = np.linalg.svd(points - np.mean(points, axis=1, keepdims=True))

# Extract the left singular vectors
left = svd[0]
```

1 2

```
# the corresponding left singular vector is the normal vector of the best-fitting
plane

left[:, -1]
```

2

```
# its dot product with the basis vectors of the plane is approximately zero

left[:, -1] @ basis
```

2

Share Cite Follow edited May 10, 2021 at 8:31 answered Jan 15, 2012 at 17:35

Mad Physicist
137 ▲11

joriki
238k ● 14 ■ 300 ▲ 525

1 @G4bri3l: I'm not sure I understand that question. The way I've defined $\mathbf{X}$, its right-singular vectors are $N$-dimensional, so I don't see how they could be used to find the best-fitting line. It's the left singular vectors that are $3$-dimensional, and indeed the left singular vector $u$ corresponding to the largest singular value gives the direction of the best-fitting line. Remember that $\mathbf{X}$ contains the coordinates with the centroid $c$ subtracted out, so the equation for the best-fitting line is $c + \lambda u$. – joriki Jan 17, 2012 at 17:18

1 Having only a normal vector is not enough to define a plane. What is the best fitting point? – xaav Nov 16, 2016 at 2:11

3 Note that if the matrix is kept as `(N, 3)`, then the normal vector will correspond to the **right** singular vector associated with the smallest eigenvalue. – Kevin Zakka Jan 7, 2020 at 11:37

1 @Siyh: OK, I've added a Python implementation to the answer. – joriki Jan 23, 2020 at 13:49

1 Anyone using Matlab can use the matgeom package, which implements this algorithm in its `fitPlane` function. – Minh Nghĩa Apr 16, 2020 at 11:20

47

A simple least squares solution should do the trick.

The equation for a plane is: $ax + by + c = z$. So set up matrices like this with all your data:

$$\begin{bmatrix} x_0 & y_0 & 1 \\ x_1 & y_1 & 1 \\ & \dots & \\ x_n & y_n & 1 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \end{bmatrix} = \begin{bmatrix} z_0 \\ z_1 \\ \dots \\ z_n \end{bmatrix}$$

In other words:

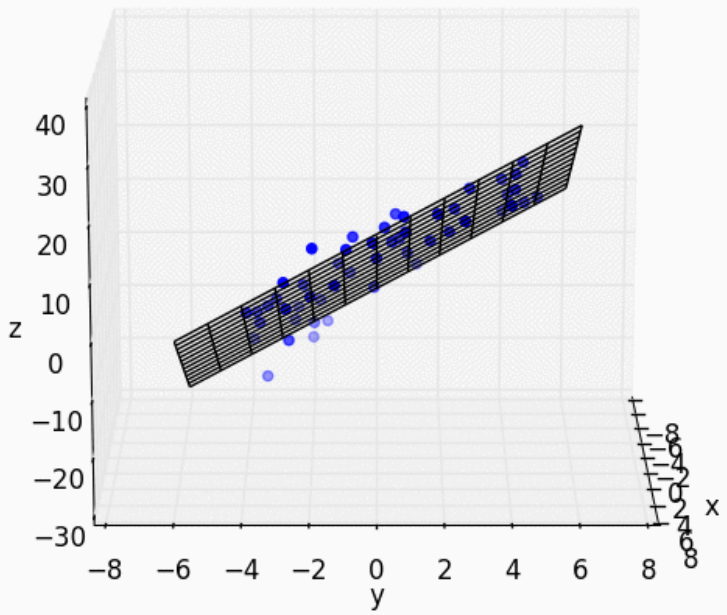
$$Ax = B$$

Now solve for $x$ which are your coefficients. But since (I assume) you have more than 3 points, the system is over-determined so you need to use the left pseudo inverse: $A^+ = (A^T A)^{-1} A^T$. So the answer is:

$$\begin{bmatrix} a \\ b \\ c \end{bmatrix} = (A^T A)^{-1} A^T B$$

And here is some simple Python code with an example:

```
# Manual solution
fit = (A.T * A).I * A.T * b
errors = b - A * fit
residual = np.linalg.norm(errors)

# Or use Scipy
# from scipy.linalg import lstsq
# fit, residual, rnk, s = lstsq(A, b)

print("solution: %f x + %f y + %f = z" % (fit[0], fit[1], fit[2]))
print("errors: \n", errors)
print("residual:", residual)

# plot plane
xlim = ax.get_xlim()
ylim = ax.get_ylim()
X,Y = np.meshgrid(np.arange(xlim[0], xlim[1]),
                  np.arange(ylim[0], ylim[1]))
Z = np.zeros(X.shape)
for r in range(X.shape[0]):
    for c in range(X.shape[1]):
        Z[r,c] = fit[0] * X[r,c] + fit[1] * Y[r,c] + fit[2]
ax.plot_wireframe(X,Y,Z, color='k')

ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
plt.show()
```




Share Cite Follow

edited Dec 7, 2022 at 10:09
Guillaume Jacquenot
103 ▲3

answered Jun 1, 2017 at 18:49
Ben
659 8 ▲7

19 Am I correct to interpret that this method will solve for the plane that minimizes the *vertical* (ie: z) distance and not the *perpendicular* distance? – Gabriel Nov 8, 2017 at 23:03

12 Yes, it minimizes vertical distance. – Ben Nov 9, 2017 at 0:00

Thank you very much for clarifying this Ben (and for the answer) If you have the time, I have a very similar question here (using Singular-Value Decomposition) that is giving me trouble. – Gabriel Nov 9, 2017 at 18:36

9 This is not a completely general solution . If the plane is (nearly) perpendicular to the z=0 plane this will fail. – wcochran Nov 25, 2019 at 19:20

How can the perpendicular distance be minimized? – Luke Hutchison Nov 20, 2020 at 12:41

4

Considering a plane of equation $Ax+By+Cz=0$ and a point of coordinates $(x_i,y_i,z_i)$, the distance is given by

$$d_i=\pm\frac{Ax_i+By_i+Cz_i}{\sqrt{A^2+B^2+C^2}}$$

and I suppose that you want to minimize

$$F=\sum_{i=1}^n d_i^2=\sum_{i=1}^n\frac{(Ax_i+By_i+Cz_i)^2}{A^2+B^2+C^2}$$

Setting $C=1$, we then need to minimize with respect to $A,B$

$$F=\sum_{i=1}^n\frac{(Ax_i+By_i+z_i)^2}{A^2+B^2+1}$$

Taking derivatives

$$F'_A=\sum_{i=1}^n\left(\frac{2x_i(Ax_i+By_i+z_i)}{A^2+B^2+1}-\frac{2A(Ax_i+By_i+z_i)^2}{(A^2+B^2+1)^2}\right)$$

$$F'_B=\sum_{i=1}^n\left(\frac{2y_i(Ax_i+By_i+z_i)}{A^2+B^2+1}-\frac{2B(Ax_i+By_i+z_i)^2}{(A^2+B^2+1)^2}\right)$$

Since we shall set these derivatives equal to $0$, the equations can be simplified to

$$\sum_{i=1}^n\left(x_i(Ax_i+By_i+z_i)-\frac{A(Ax_i+By_i+z_i)^2}{(A^2+B^2+1)}\right)=0$$

$$\sum_{i=1}^n\left(y_i(Ax_i+By_i+z_i)-\frac{B(Ax_i+By_i+z_i)^2}{(A^2+B^2+1)}\right)=0$$

whic are nonlinear with respect to the parameters $A,B$; then, good estimates are required since you will probably use Newton-Raphson for polishing the solutions.

These can be obtained making first a multilinear regression (with no intercept in your cas)

$$z = \alpha x + \beta y$$

and use $A = -\alpha$ and $B = -\beta$ for starting the iterative process. The values are given by

$$A = -\frac{\text{Sxy Syz} - \text{Sxz Syy}}{\text{Sxy}^2 - \text{Sxx Syy}} \qquad B = -\frac{\text{Sxy Sxz} - \text{Sxx Syz}}{\text{Sxy}^2 - \text{Sxx Syy}}$$

For illustration purposes, let me consider the following data

$$\begin{pmatrix} x & y & z \\ 1 & 1 & 9 \\ 1 & 2 & 14 \\ 1 & 3 & 20 \\ 2 & 1 & 11 \\ 2 & 2 & 17 \\ 2 & 3 & 23 \\ 3 & 1 & 15 \\ 3 & 2 & 20 \\ 3 & 3 & 26 \end{pmatrix}$$

The preliminary step gives $z = 2.97436x + 5.64103y$ and solving the rigorous equations converges to $A = -2.97075$, $B = -5.64702$ which are quite close to the estimates (because of small errors).

Share Cite Follow

edited Feb 12, 2016 at 4:59

answered Feb 11, 2016 at 5:17

Claude Leibovici

262k 52 115 245

1 In your example, the equation of the plane $z = Ax + By$ is carried out with respect to the criteria of the least square orthogonal distances between the points and the plane. I agree with your result. But, they are only two adjustable parameters $A, B$ because the origin $(0, 0, 0)$ is supposed to be on the plan. This is an additional condition. If we consider the more general equation of the plan $z = Ax + By + C$ the three parameters problem should lead to a better fitting with lower mean square deviation. – JJacquelin Feb 12, 2016 at 15:18

1 The general plane equation is $Ax + By + Cz + D = 0$ or $Ax + By + Cz = 1$ – John Alexiou Nov 12, 2022 at 0:19

4

In order to complete the Claude Leibovici's answer :

With the numerical example proposed by Claude Leibovici who computed the parameters of a fitted plane $z = Ax + By$, the fitting of the plane $Z = \alpha X + \beta Y + \gamma$ can be carried out thanks to the principal components method (as suggested by joriki).

The theory can be found in many books. A synopsis is given pages 24-25 in the paper: https://fr.scribd.com/doc/31477970/Regressions-et-trajectoires-3D

The symbols used below correspond to those in the formulas from the referenced paper.

| $X_k$ | $Y_k$ | $Z_k$ |
|---|---|---|
| 1 | 1 | 9 |
| 1 | 2 | 14 |
| 1 | 3 | 20 |
| 2 | 1 | 11 |
| 2 | 2 | 17 |
| 2 | 3 | 23 |
| 3 | 1 | 15 |
| 3 | 2 | 20 |
| 3 | 3 | 26 |

$Xm = 1.8 \qquad Ym = 1.8 \qquad Zm = 15.5$

$$[M]^T\,[M] = \begin{pmatrix} -0.8 & -0.8 & -0.8 & 0.2 & 0.2 & 0.2 & 1.2 & 1.2 & 1.2 \\ -0.8 & 0.2 & 1.2 & -0.8 & 0.2 & 1.2 & -0.8 & 0.2 & 1.2 \\ -6.5 & -1.5 & 4.5 & -4.5 & 1.5 & 7.5 & -0.5 & 4.5 & 10.5 \end{pmatrix} \begin{bmatrix} -0.8 & -0.8 & -6.5 \\ -0.8 & 0.2 & -1.5 \\ -0.8 & 1.2 & 4.5 \\ 0.2 & -0.8 & -4.5 \\ 0.2 & 0.2 & 1.5 \\ 0.2 & 1.2 & 7.5 \\ 1.2 & -0.8 & -0.5 \\ 1.2 & 0.2 & 4.5 \\ 1.2 & 1.2 & 10.5 \end{bmatrix} = \begin{pmatrix} 6.36 & 0.36 & 21.1 \\ 0.36 & 6.36 & 37.1 \\ 21.1 & 37.1 & 274.25 \end{pmatrix}$$

| **Plane ( $\Pi_i$ )** | $i$ | 1 | 2 | 3 |
|---|---|---|---|---|
| Eigenvalue | $\lambda_i$ | 6.0560734382 | 280.892834422 | 0.0210921398 |
| Eigenvector $\vec{V}_i$ | $V_{i,1}$ | −0.8835731892 | 0.0761187393 | 0.4620653166 |
| | $V_{i,2}$ | 0.4682691255 | 0.1336308159 | 0.8734224815 |
| | $V_{i,3}$ | 0.0047376529 | 0.9881036092 | −0.1537166616 |
| | $a_i$ | −186.5001946777 | 0.0770351799 | −3.0059546679 |
| | $b_i$ | 98.8398970657 | 0.13523968 | −5.6820286891 |
| | $A_i$ | −1.3107183496 | −0.004850442 | −21.7240278973 |
| | $B_i$ | 0.6946441368 | −0.0085152294 | −41.0640090729 |
| | $C_i$ | 0.0070279731 | −0.0629639865 | 7.2269978417 |
| | $(\Sigma\delta_k^2)_i$ | 6.0560734382 | 280.892834422 | 0.0210921398 |

Equation of the selected plane : $A\,X + B\,Y + C\,Z + 1 = 0$

$$\begin{cases} A = -21.7240278973 \\ B = -41.0640090729 \\ C = 7.2269978417 \end{cases}$$

$$Z = \alpha X + \beta Y + \gamma \qquad \begin{aligned} \alpha &= 3.0059546679 \\ \beta &= 5.6820286891 \\ \gamma &= -0.1383700427 \end{aligned}$$

Share Cite Follow

answered Feb 12, 2016 at 18:48

JJacquelin

66.3k ● 3 ■ 37 ▲ 88

---

0

All the other answers are amazing. I am providing an another way of doing this. You can use RANSAC. This method may not be more efficient than the other methods.

Simple theory behind this method is to pick random samples from the input sample, and create mathematical model that better describes your problem. After that find the error of this mathematical model with respect to all the samples, and count the number of samples whose error is within a given threshold (These samples are called inliers) . Do this procedure for multiple iterations and return the model that has more number of inliers.

For your specific problem, at a give iteration , you can pick three random points, and construct a plane using those points. The generalized equation of a plane is as given by $Ax + By + Cz + D = 0$. Now for every point of coordinates $(x_i, y_i, z_i)$ in the input sample , find the distance between point and the plane. This can be your error definition. The

distance between the plane and the point is given by

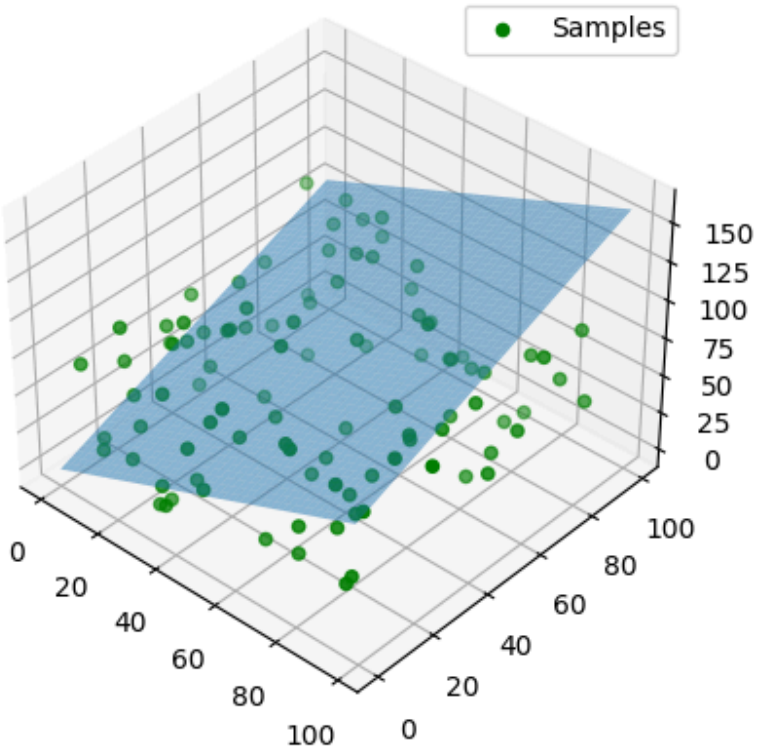
$$d_i = \pm \frac{Ax_i + By_i + Cz_i + D}{\sqrt{A^2 + B^2 + C^2}}$$

These equations are taken from [Claude's](#) answer.

If this value is less than a given threshold, you can count that point as an inlier. Repeat this process for certain number of iterations. The following python program implements this method.

```
        # print(A,B,C,D)
        inliers= 0
        #Now for every point in input sample, find if it is a inlier
        for p in X:
            xi,yi,zi = p

            d = np.abs((A*xi+B*yi+C*zi+D)/np.sqrt(xi**2+yi**2+zi**2))
            if d<threshold:
                inliers+=1 # finding out the inliers that satisfy the distance
condition
        if inliers > n:
            Abest = A
            Bbest = B
            Cbest = C
            Dbest = D

    if inliers > 0:
        return (Abest , Bbest, Cbest, Dbest)
    return None


if __name__ == "__main__":
    rdm = np.random.RandomState(10)  # to repeat the random state
    X = rdm.randint(100,size=(100,3))
    fig = plt.figure()
    ax = plt.axes(projection='3d')
    Out = RANSAC(X)
    if Out:
        A,B,C,D = Out
        ax.scatter3D(X[:,0],X[:,1],X[:,2],c="g",label="Samples")
        xx, yy = np.meshgrid(range(100), range(100))
        z = -(A*xx+B*yy+D)/C
        ax.plot_surface(xx, yy, z, alpha=0.5) #,label = "Best Fit Plane ")
        plt.legend()
        plt.show()

    else:
        print("No best RANSAC")
```

This program results the following for a random input :




The problems with this method :

1. May need tuning for iterations and threshold
2. Since the random points are taken to form the mathematical model , this may not give you the best ouput
3. This is slow !

Share Cite Follow

answered Nov 11, 2022 at 22:30

McLovin
133 ▲5

**Highly active question**. Earn 10 reputation (not counting the association bonus) in order to answer this question. The reputation requirement helps protect this question from spam and non-answer activity.